

Page No:			
Date:			

- CI/CD Workflow in Github only
- NO need for extra manual setup [like Jenkins]
- Automate tasks: Testing, building, deployment & notification
- Supports all language, all platform.
- Huge marketplace for inbuild action [Just Import & All set!]
- Scalable runners. Using action.

[Note - ~~Artifact~~ Artifact created in the workflow is destroyed as soon as CI/CD workflow is completed/deployed. To store it → Include jobs to store to ECR, S3 etc]

- Automated on trigger $\rightarrow$ very simple

Core Concepts:

Workflow $\rightarrow$ Automated CI/CD process.
Job - Set of steps, runs on runner

Step- single task [run script /
via action]

Action - Import / Reusable piece of automation

Runner - where jobs run [server]

Notes:

→ Every workflow must be saved in:-

.github/workflows/ci-cd.yaml
 Folder-1 Folder-2 File

→ Keywords: ie Syntax

- on → Trigger [on what trigger run the workflow]

[Push, pull, schedule, workflow_dispatch]

- env → Global variables

Use it → \${{env.variable_name}}

→ same for [secrets.variable_name]

- runs-on: Choose OS & VM types

- needs: {job-name}

[It require mentioned job name to be executed completely, in order for the given job to run]

- steps:

- name: [step 1]

- name: [step 2]

- run: [shell command]

- run: |

-- command-1

-- command-2

} multiline
command

} uses: action

- To run

reusable with

action

[Pair with]

} with:

node-version: 18

username:

password:

[with require
user]

Structure

Page No:

Date:

name: CI-CD pipeline

on:

push:

branches: [main]

jobs:

job-name:

~~no type st~~

runs-on: ubuntu-latest

steps:

- name: step name

action uses: actions/checkout@v4

- name: step name

run: command

job-2:

runs-on:

needs:

steps:

Full GitHub Action Workflow [Builds Deploy]

Builds Deploy

Page No. 888 ECR -> EAS

Date.

--	--	--

name: CI-CD pipeline

Code



on:

Builds

push:



branch: [main]

Deploy

env:

jobs:

AWS_REGION: ap-south-1

ECR_REPOSITORY: my app

IMAGE_NAME: my app

jobs:

build-test:

name: Build & Test (Maven)

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v1

- name: set up JDK

uses: actions/setup-java@v1

with:

distribution: temurin

java-version: 17

- name: Build with Maven

run: mvn clean package

- name: Run test

run: mvn test

[Job-2]
=

build-push:

name: Build & push to Docker

runs-on: ubuntu-latest

needs: build-test

steps:

- name: Checkout code

uses: actions/checkout@v2

- name: Configure AWS

uses: aws-actions/configure-aws-credentials@v1

with:

roles-to-assume: \${{ secrets.AWS_ROLE }}

aws-region: \${{ env.AWS_REGION }}

- name: Login to ECR

uses: aws-actions/amazon-ecr-login@v1

- name: Build & push Docker image

env:

ECR-REGISTRY: \$

\${{ steps.login-ecr.outputs.registry }}

run: |

docker build -t

\$ECR-REGISTRY/\${{ env.ECR_REPO }}

: \${{ github.sha }}y

docker push \$[same]

Tagging

tag image

[Job-3]

deploy:

name: Deploy to K8s [first time]

runs-on: ubuntu-latest

needs: build-pub

steps:

- name: Check out code

uses: actions/checkout@v4

- name: Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v4

with:

role-to-assume: \${ secrets.AWS_ROLEARN }

aws-region: \${ env.AWS_REGION }

- name: Update kubeconfig

run: |

aws eks update-kubeconfig \

--name \${ secrets.EKS_CLUSTER_NAME }

\ --region \${ env.AWS_REGION }

- name: Set ie update new image in

run: |

sed -i 's/IMAGE-PLACEHOLDER/ \$

{ env.ECR_REPO } : \${ secrets.sha } | g'

K8s/deployment.yaml

- name: Apply Kubernetes manifest

run: |

kubectl apply -f K8s/

kubectl rollout status deployment/myapp

Git+OPS

- Everything in code.
- Managed by GitHub.
- If it's not in Git, it doesn't exist.
- GitOps is a subset of DevOps.

Git + Ops
↑ ↑
Versioning + Operational Tasks

- It restricts access to Infrastructure by only accessing GitHub ~~not~~ not the tools
- But Git is only a version-control system with help of GitHub tool like GitHub Actions, GitLab, ArgoCD, Tekton, Jenkins, you can manage operational tasks
- Restricting Access to Git only.

GitOps Workflow:-

1.] Write & Push code to Git
 ↓ source code + YAML files. [Github, Gitlab]

2.] CI Tools [Github Action, Gitlab CI, Jenkins]

- Run Test
- Build Docker image
- Pushes to registry
- Updates the deployment - YAML files
- Commit to repository

↓

3.] GitOps controller [Argo CD, Flux]

- Continuously monitors changes in repo
- As soon as a change is detected, automatically applies it to K8s cluster

↓

4.] Kubernetes Environment [AKS, EKS, GKE]

- where actual cluster runs

→ Auto sync, Drift Detection & Rollback

- If someone manually changes a configuration not in the github repo

- ArgoCD detects it

- Heals back to originally configuration in the existing repo

GitOps Project Architecture

Page No:

104

Date:

